Travel Planning Agent

Internal Assessment Task II - Agentic AI System using Modern LLM-based Tools

16010123001: Aarav Shah

16010123002: Diya Mehta

16010123003: Kabir Rao

Introduction, Abstract of problem statement

Planning a trip requires collecting information about places, costs and weather from many websites. We built an agent that takes a travel goal from the user and produces a day-wise itinerary within a budget.

Manual trip planning is slow and error-prone because budgets, prices and weather must be checked together. Our agent turns a natural-language travel goal into a budget-checked itinerary and shows every step it took.

AI framework(s) used

We used a ReAct-style agent built with LangChain, with Llama 3.3 70B accessed through the OpenRouter free API. LangChain provides the tool-calling loop and OpenRouter gives free access to an open model.

Tools integrated with agent

The agent uses three tools: web_search (Tavily search API) for places and prices, calculator for budget arithmetic, and weather_api (Open-Meteo) for the forecast. Each tool was chosen because it supplies information the LLM cannot know reliably on its own.

Part A - Agent Design (Modern View)

Goal of the agent

The goal is to produce a complete travel itinerary that respects the user's budget and dates.

Agent role

The agent acts as a travel planner that searches, compares, calculates and plans.

Tools used

web_search, calculator and weather_api as described above.

Memory type

Short-term conversational memory (the ReAct scratchpad of thoughts and observations) is kept in the context window; user preferences such as budget are stored in a JSON file between sessions.

Planning strategy

The agent follows the ReAct strategy: it reasons about the goal, calls a tool, observes the result and decides the next step until the itinerary is complete.

Part B - Implementation (Modern AI Tools)

LLM-based agent

Llama 3.3 70B was chosen because it is free on OpenRouter, supports long context and follows tool-calling instructions well.

Architecture diagram

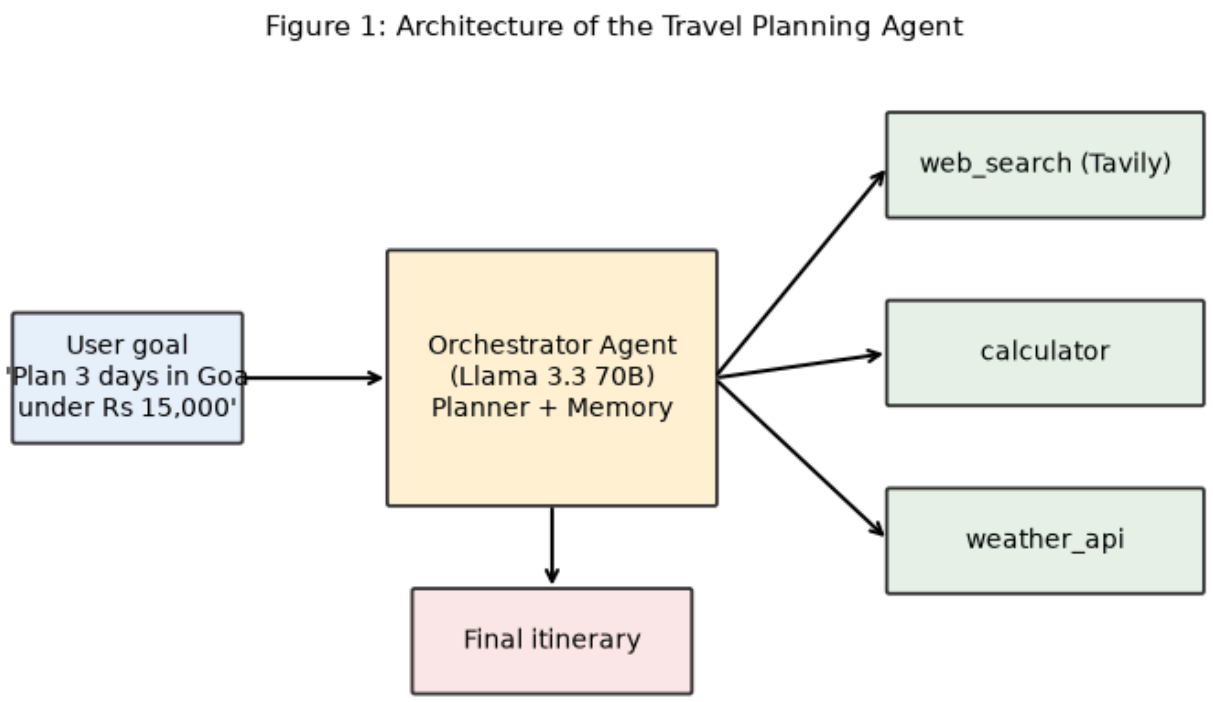


Figure 1: Architecture of the Travel Planning Agent

Tool flow diagram

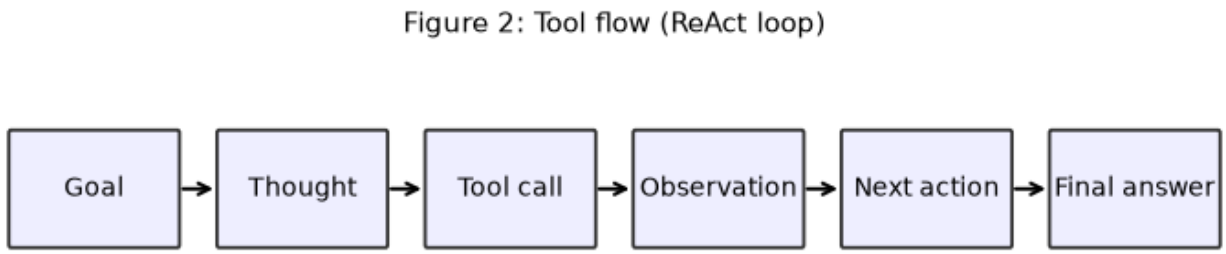


Figure 2: Tool flow (ReAct loop)

Agent interaction log

```
[Thought] I need hotel prices and weather first.
[Tool Call] web_search('budget hotels north goa price per night')
[Observation] 3 results: Rs 1800-2500 per night
[Tool Call] weather_api('Goa', '2026-12-10')
[Observation] Sunny, 31C
[Tool Call] calculator('3*2200 + 4500 + 2000')
[Observation] 13100
[Final Answer] Day 1: Baga beach ... total Rs 13,100
```

Prompts and output screenshots



Screenshot 1: Prompt and agent output

Code repository

Code and notebook: https://github.com/example-group/travel-planning-agent

Part C - Analysis & Comparison

Classical Agent (Syllabus)	**Modern Agent (This Activity)**
Rule-based	LLM-driven
Static logic	Dynamic planning
No memory	Contextual memory
Single-step	Multi-step reasoning

A rule-based planner can only follow fixed if-then rules for known cities, while our agent plans dynamically and adapts when a tool returns unexpected results.

Discussion, takeaway

Tool use made the itinerary costs reliable; the main limitation is the rate limit of free APIs and occasional malformed tool calls, which we handle with retries.

Conclusion

The agent plans trips end to end with observable reasoning and verified budget calculations.

References

[1] S. Yao et al., ReAct: Synergizing Reasoning and Acting in Language Models, ICLR 2023.

[2] LangChain documentation, https://python.langchain.com